COSC3106: Theory of Computing

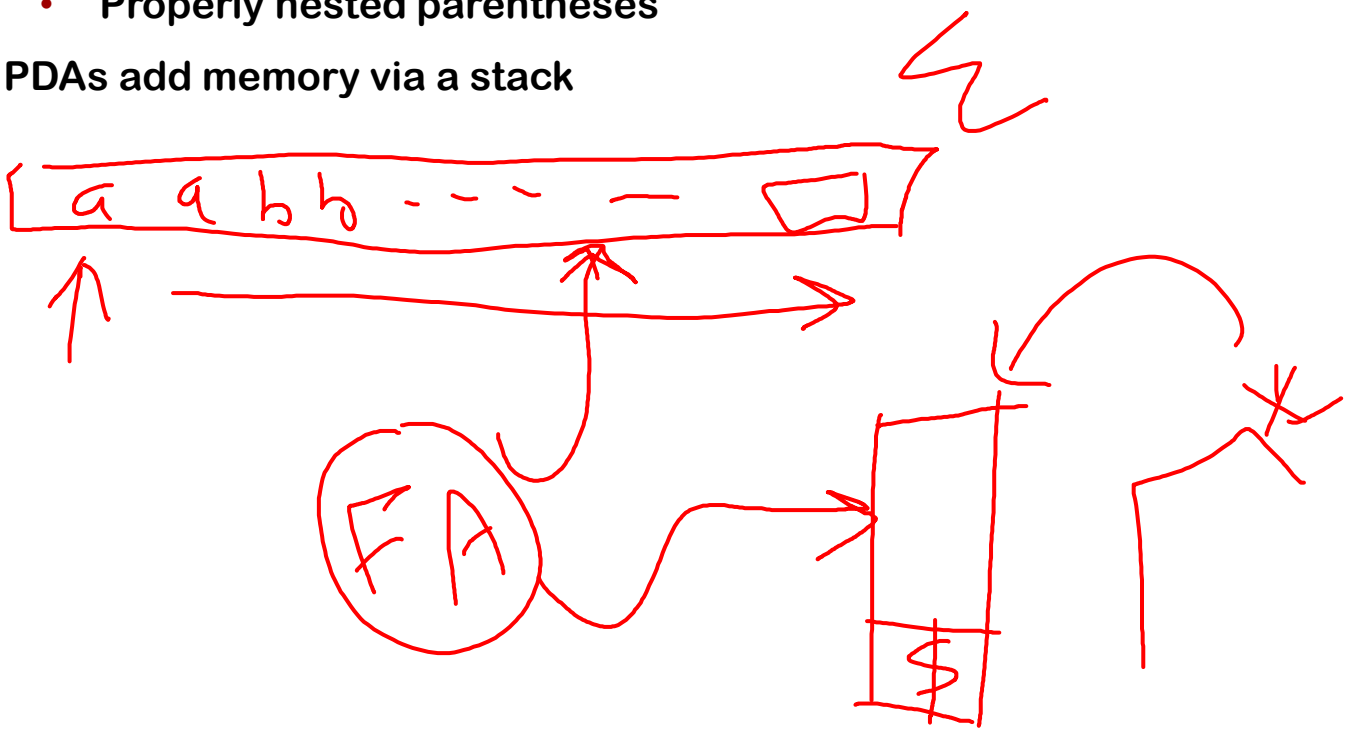
Lec#12: Pushdown Automata (PDA)

School of Computer Science and Technology

Algoma UNIVERSITY

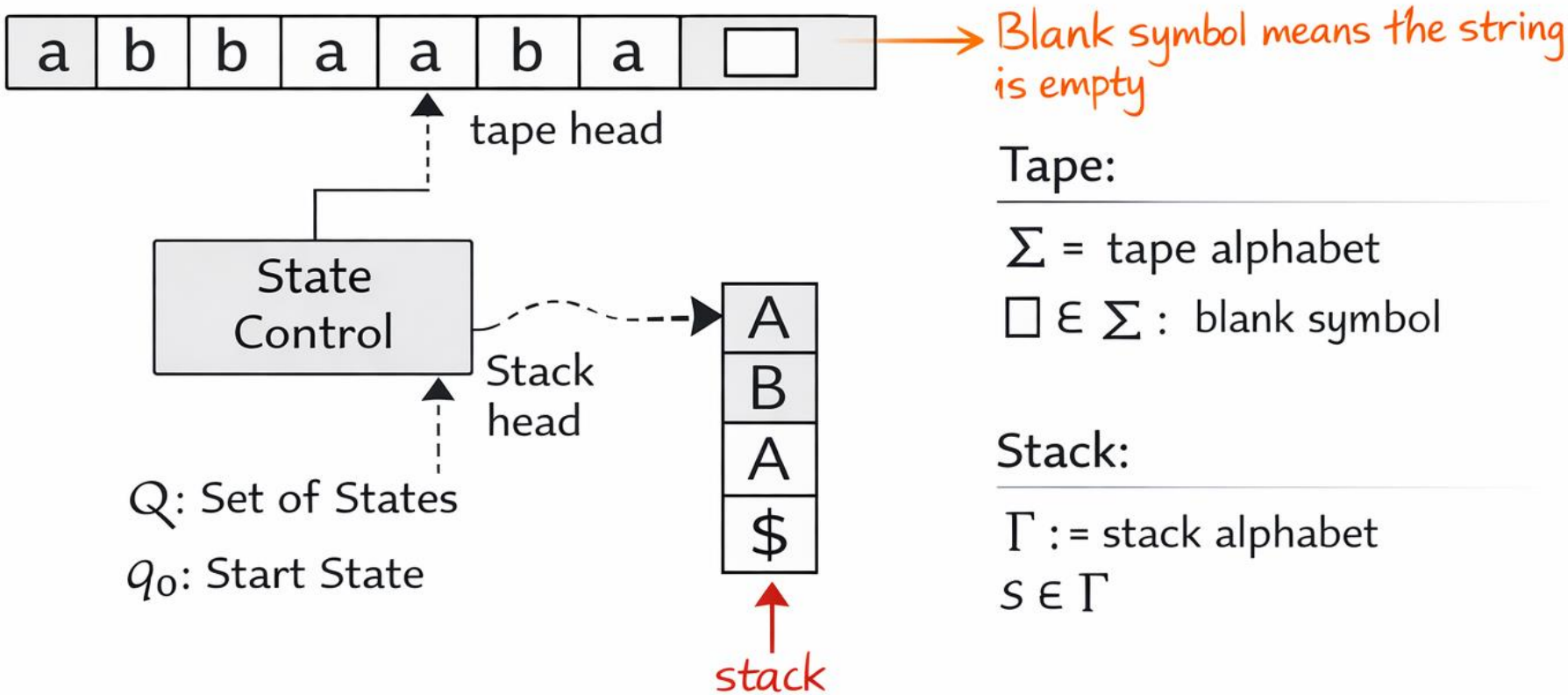
- In this lecture, you will…
  - Explain the limitations of finite automata
  - Describe why a stack is needed for recognizing certain languages
  - Define what a Pushdown Automaton (PDA) is

- **Why do we need PDAs?**
  - Finite automata cannot recognize:
    - $L = \{w = a^n b^n : n \geq 0\}$.
    - Properly nested parentheses
  - PDAs add memory via a stack



Algoma UNIVERSITY

PDA Components: A PDA consists of:



- **Tape (Input string): Σ={a,b}**
- **Tape head (reads symbols)**
- **State control**
- **Stack (infinite memory in LIFO form): Γ={$,S}**

Algoma UNIVERSITY

Formal Definition of PDA:

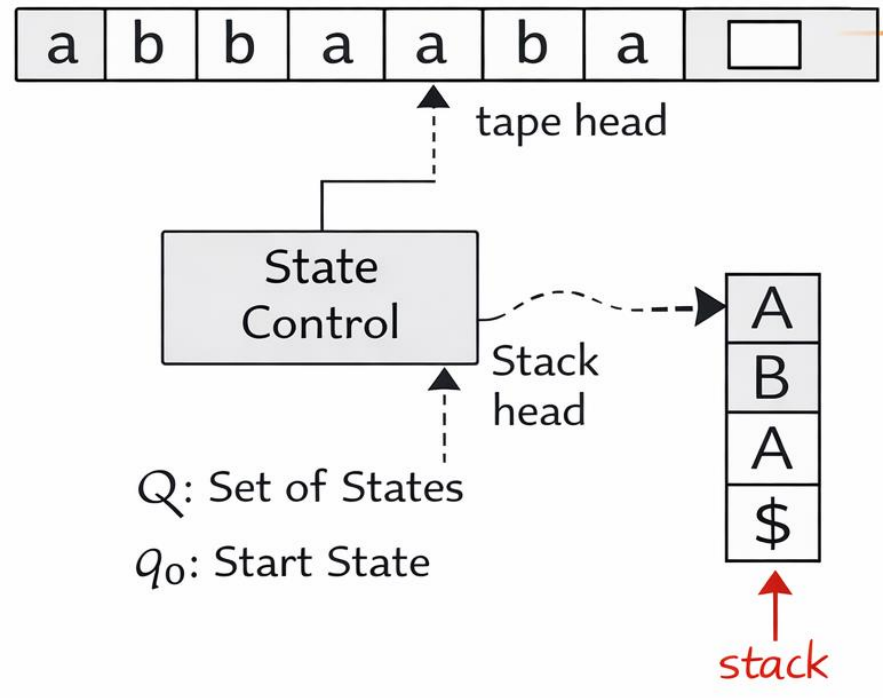
- A PDA is a 5-tuple:

$$M = (Q, \Sigma, \Gamma, \delta, q_0)$$

Where:

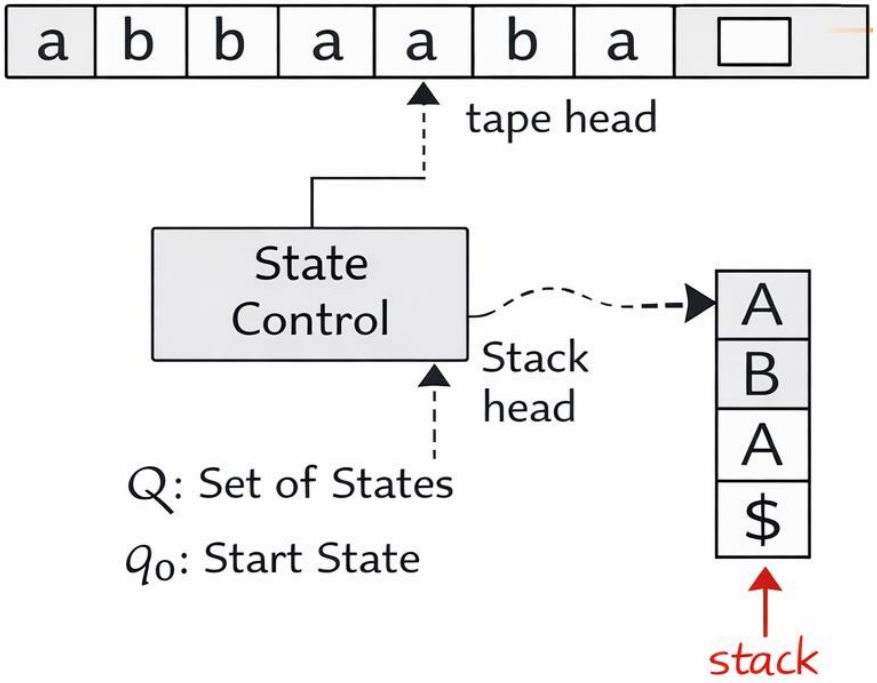
- Q: set of states
- Σ: input alphabet
- Γ: stack alphabet
- δ: transition function
- q_0 :start state

- $\delta: Q \times (\Sigma \cup \{\square\}) \times \Gamma \rightarrow Q \times \{R, N\} \times \Gamma^*$



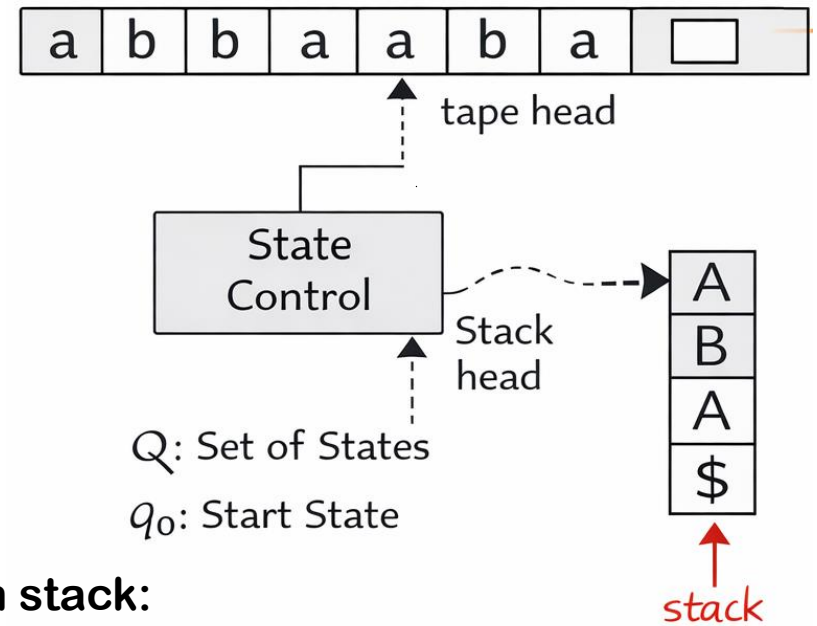
Algoma UNIVERSITY

Transition Function:



- δ: $Q \times (\Sigma \cup \{\square\}) \times \Gamma \rightarrow Q \times \{R, N\} \times \Gamma^*$
  - **Meaning:**
    - **Current state**
    - **Input symbol (or blank)**
    - **Top of stack**
  - **Produces:**
    - **New state**
    - **Move: Right (R) or No move (N)**
    - **Update the stack**

Transition Function:



- δ: $Q \times (\Sigma \cup \{\square\}) \times \Gamma \rightarrow Q \times \{R, N\} \times \Gamma^*$

- $\delta(r, a, A) = (r', R \text{ or } N, w)$

➢ **Meaning:**

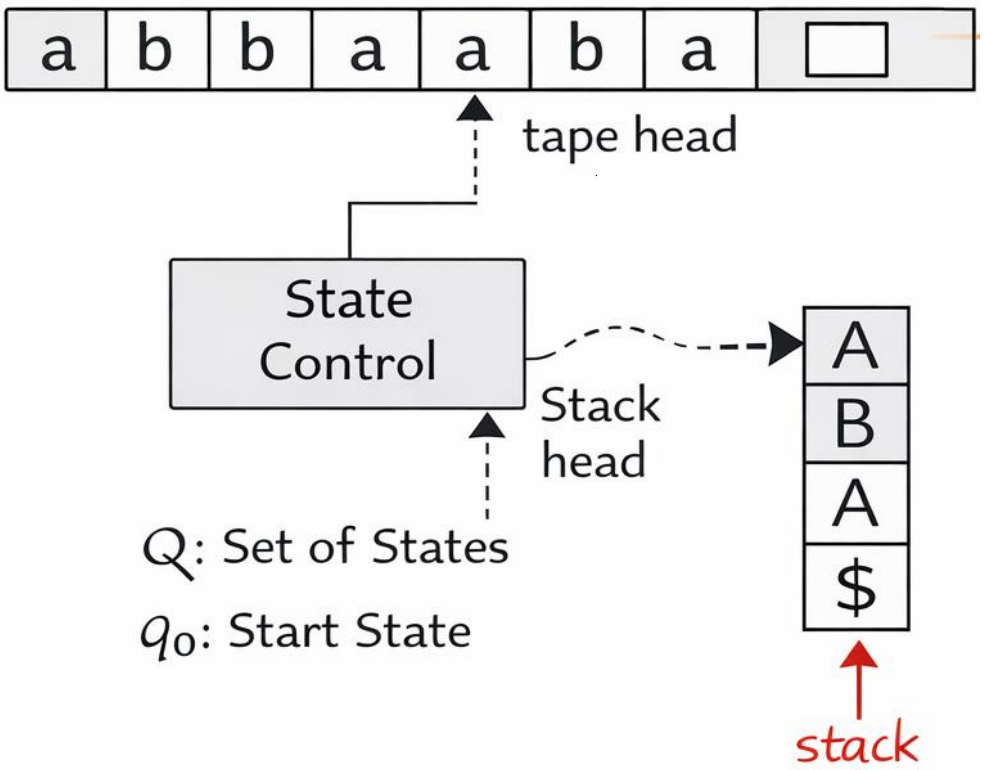
- In state r, reading $a \in \Sigma$, with $A \in \Gamma^*$ on stack:
- Move to state r'
- Move right (or not move)
- Replace A with string w.

➢ **Instruction:**

- ➢ $r\ a\ A = r'\ R\ w$
- ➢ $r\ a\ A = r'\ N\ w$

Algoma UNIVERSITY

- **Stack Operations:**
  - If $w = \epsilon$
    - **Pop top of stack**
  - If $w = B_1 B_2 B_3 \ldots B_k$
    - **Push sequence onto stack**



Algoma UNIVERSITY

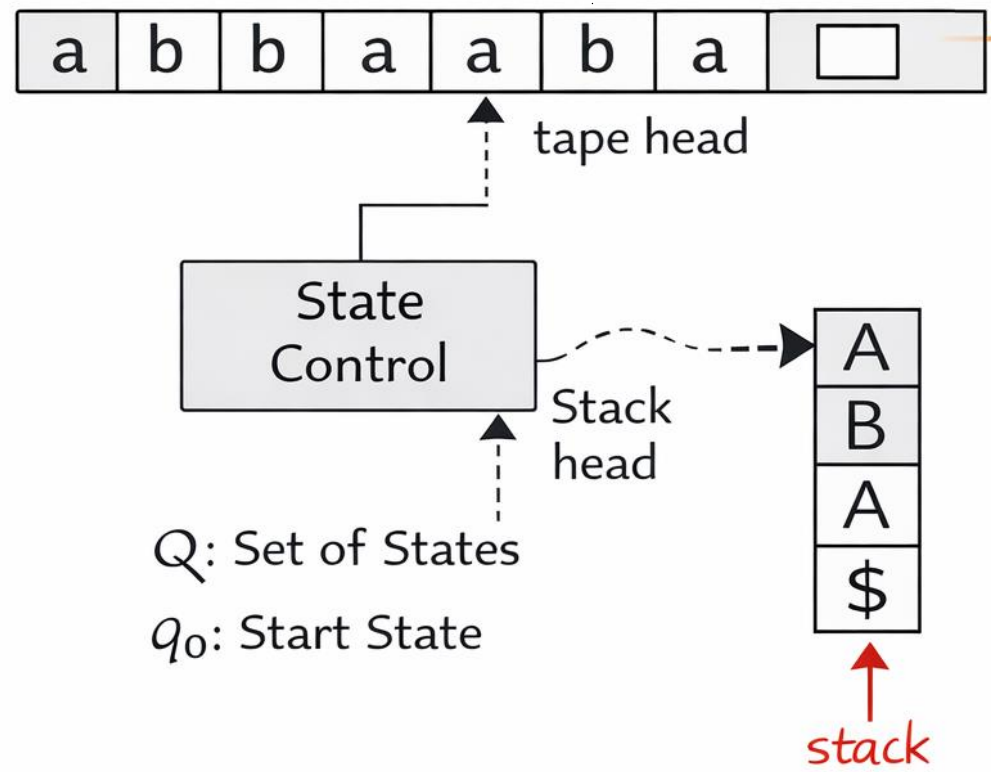
Computation of a PDA:

- Now we need to specify what does it means the PDA that it makes a computation:

- Given input string:

$$w = a_1\, a_2\, a_3\, \ldots\, a_n$$

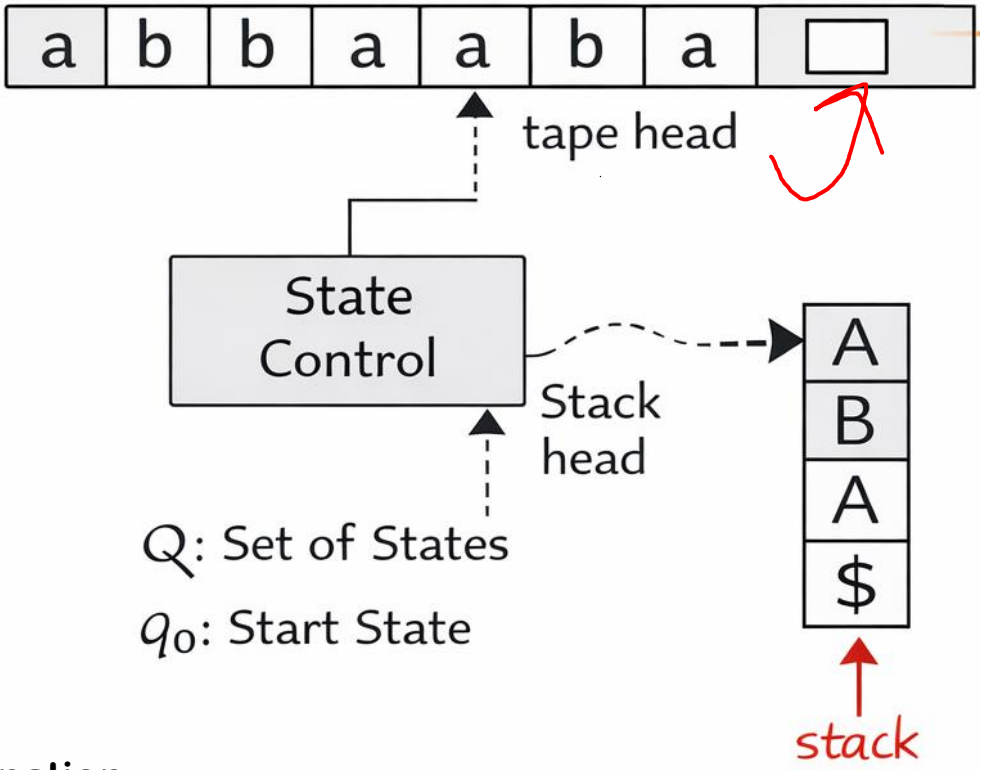
- Steps:
  1. Start at q_0
  2. Read input left to right
  3. Apply transitions
  4. Update stack



Algoma UNIVERSITY

Computation of a PDA:

- Acceptance Condition:
  - A PDA accepts a string if:
    - It terminates, and
    - The stack becomes empty
- Rejection Conditions:
  - Reject if:
    - Does not terminate
    - Stack is not empty at termination



Algoma UNIVERSITY

Properly Nested Parentheses:

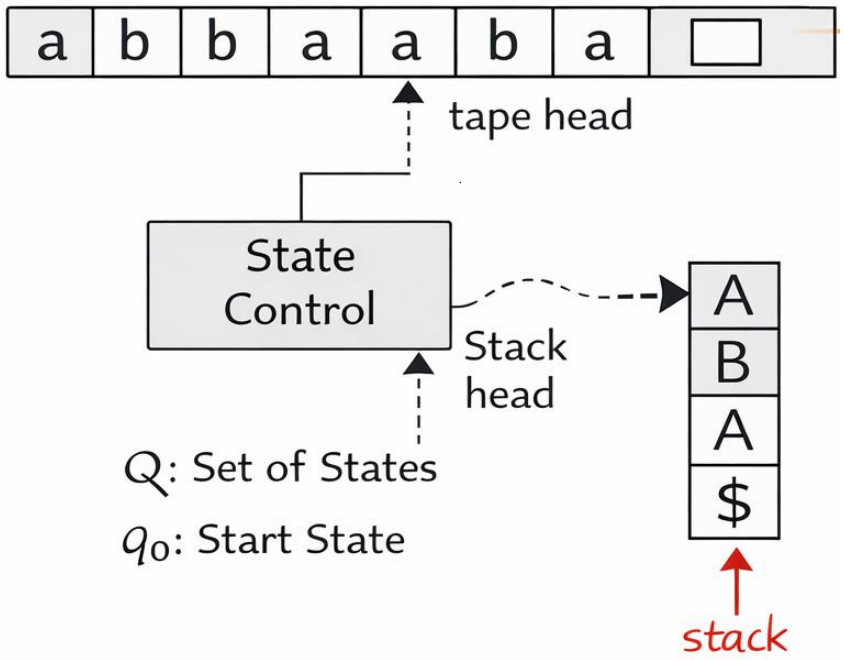
- Recognize:
  - Properly nested parentheses
  - Ex:
    - (())() → Accept ✔ → $aabbab$
    - (()) → Reject ✘ → $aabbb$

- Language Encoding:

  Let:
  - $a = ($
  - $b =)$

  - Goal: To ensure:
    - #a ≥ #b at all prefixes
    - Total #a = #b

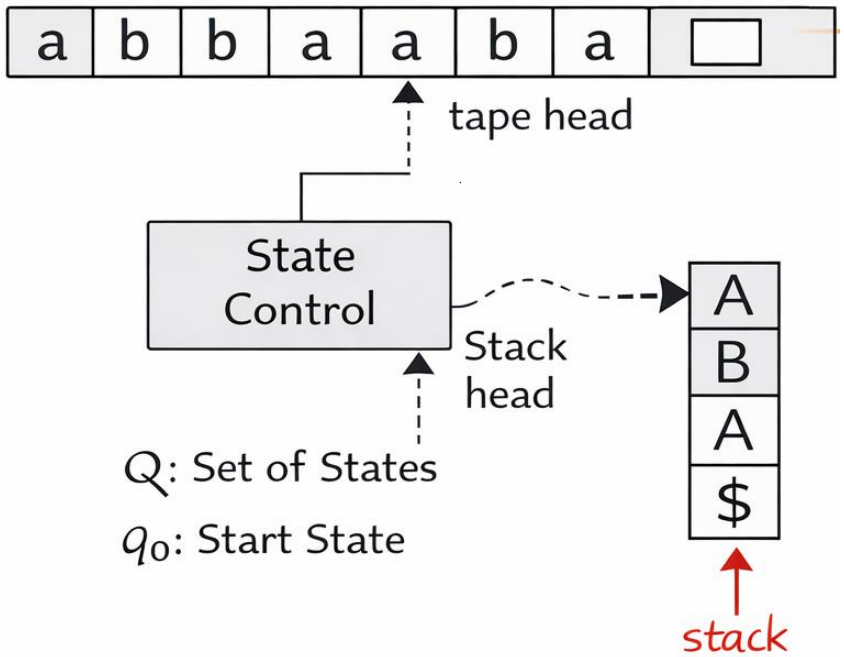
- **PDA Strategy:**
  - **Algorithm:**
    - **Read input left → right**
    - **For each:**
      - a: **push** S
      - b: **pop** S



- **Stack Meaning:**
  - **Stack stores:** $\#a$ **read so far** $-$ $\#b$ **read so far**

Algoma UNIVERSITY

PDA Transition Rules:



1. **Push for a:**
   - $\delta(q, a, \$) = (q, R, \$S)$
   - $\delta(q, a, S) = (q, R, SS)$
2. **Pop for b:**
   - $\delta(q, b, S) = (q, R, \epsilon)$
3. **Invalid Case:**
   - $\delta(q, b, \$) = (q, N, \epsilon)$: **Means more b than a → Reject**
4. **Acceptance Case:**
   - $\delta(q, \quad, \$) = (q, N, \epsilon)$: **If input finished and stack is empty → Accept**

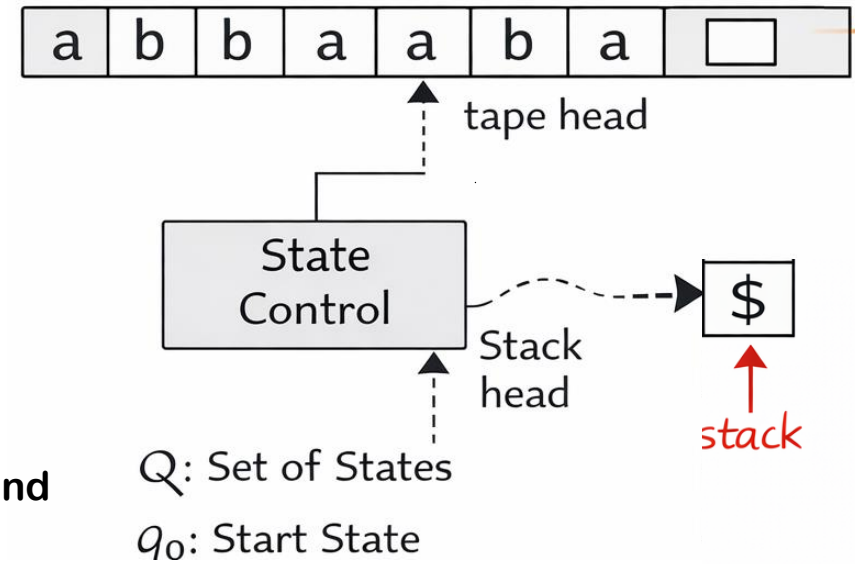
Algoma UNIVERSITY

- **Ex#1: Target Language**

$$L = \{a^n b^n \mid n \geq 0\}$$

  - **Show PDA for this language?**

  - **We split the computation into two phases:**
    - **Reading a's → push S onto stack**
    - **Reading b's → pop S from stack**
    - **Accept if stack becomes empty at the end**

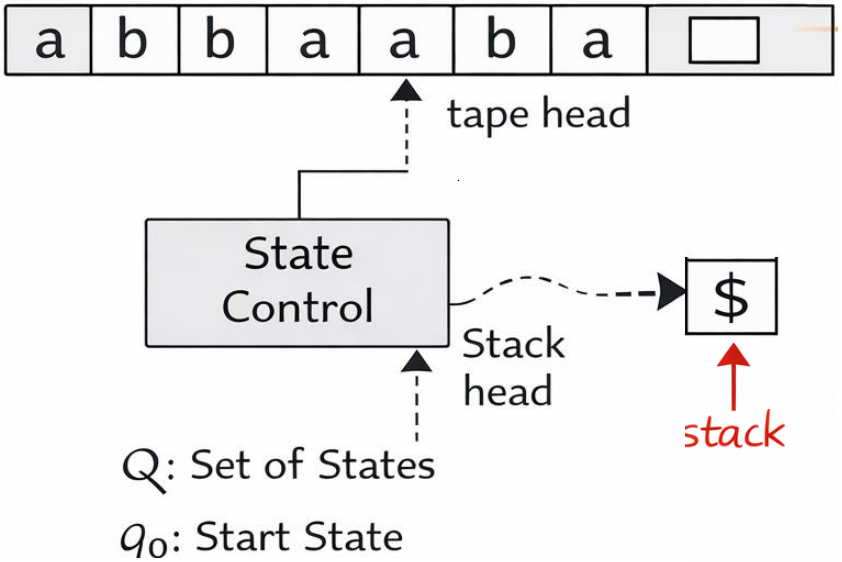


- **States Definition:**
  - q_a : **reading a's**
  - q_b : **reading b's**

Algoma UNIVERSITY

- **Ex#1: Target Language**

$$L = \{a^n b^n \mid n \geq 0\}$$



- **Behavior of States**

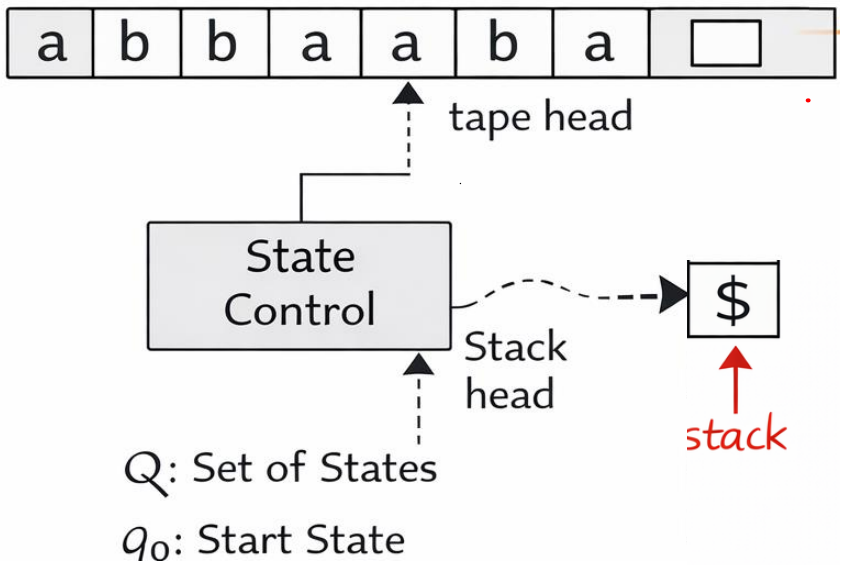
  In q_a:
  - Still reading a's
  - Push symbols onto stack

  In q_b:
  - Reading b's
  - Pop from stack
  - No more a's allowed

Algoma UNIVERSITY

- **Ex#1**: Target Language

$$L = \{a^n b^n \mid n \geq 0\}$$



- **Transition:**
  - **Push S for every a:**

$$\delta(q_a, a, \$) = (q_a, R, \$S)$$
$$\delta(q_a, a, S) = (q_a, R, SS)$$

  - **First b: start popping:**

$$\delta(q_a, b, S) = (q_b, R, \epsilon)$$

  - **Pop S for every b:**

$$\delta(q_b, b, S) = (q_b, R, \epsilon)$$

  - **Acceptance Condition: (Input finished + stack empty → Accept)**

$$\delta(q_b, \square, \$) = (q_b, N, \epsilon)$$

  - **Reject:**
    - **If b appears when stack has only $**

$$\delta(q_a, b, \$) = (q_{\text{rej}}, N, \$)$$
$$\delta(q_b, b, \$) = (q_{\text{rej}}, N, \$)$$

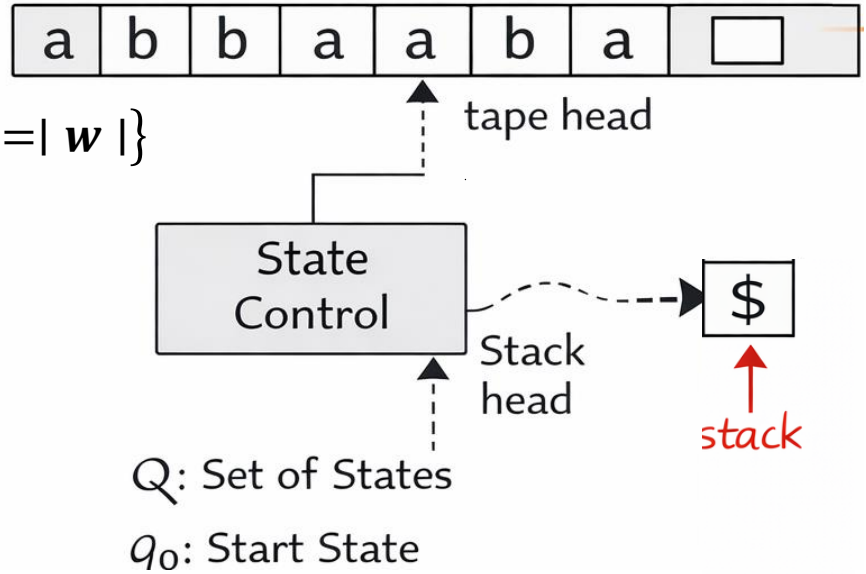
    - **If a appears in state q_b**

$$\delta(q_b, a, S) = (q_{\text{rej}}, N, S) \qquad \delta(q_a, \square, S) = (q_{\text{rej}}, N, S)$$
$$\delta(q_b, a, \$) = (q_{\text{rej}}, N, \$)$$

Algoma UNIVERSITY

Ex#2: Target Language

$$L = \{vbw \mid v, w \in \{a, b\},\ \ |v| = |w|\}$$



- Formal PDA Definition
  - $Q = \{q, q'\}$
  - $\Sigma = \{a, b\}$
  - $\Gamma = \{\$, S\}$
  - Start state: q
  - Initial stack: $\$$
- State q reading v (push) or switch to q'
- State q' reading w (pop) and check if $|w| = |v|$

Algoma UNIVERSITY